

Reviewer Pack — Minimal Geometric Flows Energy and Resolution Proof Width Co...

Entry 03e4bd003029 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 22:05:55 UTC

Minimal Geometric Flows Energy and Resolution Proof Width Correlation

Entry ID: 03e4bd003029 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 22:05:55 UTC

1. Conjecture

Field A (mathematical branch): Differential Geometry (Geometric Flows) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

The minimal geometric flows energy (MGE) of a Boolean satisfiability problem instance is linearly correlated with its resolution proof width, such that $MGE(\varphi) = \Theta(w(\varphi))$ for any Tseitin formula φ .

Rationale (proposer's reasoning):

Geometric flows provide a means to analyze the complexity of computation by examining the evolution of geometric structures over time. By mapping Boolean satisfiability problems onto these geometric spaces and analyzing the energy associated with their flows, we may uncover new structural insights into resolution proof complexity.

Taxonomy category: geometric_flows_resolution (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cae00a3a330c1c63

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between $MGE(\varphi)$ and $w(\varphi)$ across all 30 seeds is greater than or equal to 0.7, with no seed producing a correlation coefficient less than 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal geometric flows" AND "resolution proof width" - "Boolean satisfiability problem" AND "geometric flows energy" - "Tseitin formula" AND resolution proof complexity

Top relevant hits considered: - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/2006.04124v1] On the Complexity of Branching Proofs - [http://arxiv.org/abs/1004.2159v2] Algebraic Proofs over Non-commutative Formulas - [s2:10.4230/LIPIcs.MFCS.2019.49] Bounded-depth Frege complexity of Tseitin formulas for all graphs - [s2:2103.09609] Characterizing Tseitin-formulas with short regular resolution refutations

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        if n <= 0:
            return ""
        variables = [f"x{i}" for i in range(1, n+1)]
        clauses = []
        for i in range(1, n+1):
            clauses.append(f"{variables[i-1]}")
        for i in range(1, n+1):
            for j in range(i+1, n+1):
                clauses.append(f"~{variables[i-1]} | ~{variables[j-1]}")
        return " & ".join(clauses)

    def calculate_mge(phi):
        # Placeholder function to compute MGE
        # In practice, this would involve complex geometric calculations
        return random.uniform(0.5, 2.0) * len(phi.split(" & "))

    def calculate_w(phi):
        # Placeholder function to compute resolution proof width
        # In practice, this would involve a DPLL solver
        return random.randint(1, 10)
```

```

n = random.choice([5, 10, 15, 20, 30, 40])
phi = generate_tseitin_formula(n)
mge = calculate_mge(phi)
w = calculate_w(phi)

return {
    "metric_name": "MGE",
    "metric_value": mge,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    if not sys.argv[1:]:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79]
    else:
        seeds = [int(s) for s in sys.argv[1:]]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] and r["metric_value"] < 0.5 for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"] and r["metric_value"] < 0.5)
        print(f"RESULT: FALSIFIED counterexample='low_correlation' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

64.9581749134122, 'instances_tested': 1, 'n_max': 40, 'conjecture_holds': False, 'counterexample': '
TRIAL: {'metric_name': 'MGE', 'metric_value': 92.04154045429097, 'instances_tested': 1, 'n_max': 10,
TRIAL: {'metric_name': 'MGE', 'metric_value': 40.01961049959957, 'instances_tested': 1, 'n_max': 10,
TRIAL: {'metric_name': 'MGE', 'metric_value': 9.188341560399481, 'instances_tested': 1, 'n_max': 5,
TRIAL: {'metric_name': 'MGE', 'metric_value': 126.05905742952068, 'instances_tested': 1, 'n_max': 15
TRIAL: {'metric_name': 'MGE', 'metric_value': 121.05213830208069, 'instances_tested': 1, 'n_max': 15
TRIAL: {'metric_name': 'MGE', 'metric_value': 491.98259816820337, 'instances_tested': 1, 'n_max': 40
TRIAL: {'metric_name': 'MGE', 'metric_value': 895.5154682220209, 'instances_tested': 1, 'n_max': 30,
TRIAL: {'metric_name': 'MGE', 'metric_value': 68.4292813051405, 'instances_tested': 1, 'n_max': 10,
TRIAL: {'metric_name': 'MGE', 'metric_value': 684.9349585461116, 'instances_tested': 1, 'n_max': 30,
TRIAL: {'metric_name': 'MGE', 'metric_value': 92.67064794935658, 'instances_tested': 1, 'n_max': 10,

```

TRIAL: {'metric_name': 'MGE', 'metric_value': 71.28754372162108, 'instances_tested': 1, 'n_max': 10,
RESULT: INCONCLUSIVE support_fraction=0.0

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses placeholder functions for calculating the MGE and resolution proof width, which are randomly assigned values. This does not provide a meaningful empirical test of the conjecture as it does not compute these metrics correctly.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test code uses placeholder functions for calculating the MGE and resolution proof width, which are randomly assigned values. This does not provide | next: Develop a proper implementation to calculate the MGE and resolution proof width, then retest the conjecture with accurate data.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14751
2	preregistration	ollama_remote	glm4:latest	0	0	9355
3	novelty	ollama_remote	glm4:latest	0	0	11114
4	novelty	ollama_remote	glm4:latest	0	0	9548
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42395
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14082
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10182
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9112
9	critic	ollama_remote	glm4:latest	0	0	17227
10	judge	ollama_remote	glm4:latest	0	0	9307

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 147073 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/03e4bd003029.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in [research/audit/03e4bd003029.jsonl](#) for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: [research/pvsnp_sandbox/test_*.py](#) (per-cycle)

Replay tarball: [research/replay/03e4bd003029.tar.gz](#) (if generated)